

Riderzpro EV Bike Battery MVP

Day 1 execution plan, project audit, individual ownership, branch rules, and a coordinated website + mobile build path

Project snapshot: the current Git commit contains a substantial responsive web prototype for customer, vendor and technician journeys. It uses browser-side mock data. No mobile client or implemented backend/API appears in the inspected commit tree. Day 1 should build on and assess the prototype while agreeing contracts and parallel work for a real website and mobile product.

What is present and what it means

Area	Observed in inspected commit	Implication
Web app	Vite-served vanilla JS/CSS; customer, vendor, technician role views; responsive preview; booking/tracking UI.	Keep as web baseline and demo; audit simulated behavior vs real capability.
Data/actions	Browser store and seeded bikes, batteries, fitment, slots, orders, technicians, pricing; actions mutate local state.	Agree contracts and replace mock sources behind clear module boundaries. No shared persistence today.
Backend/mobile	No backend/API implementation or Android/iOS app files in the inspected tree. README architecture describes plans, not code present.	Plan as new work; do not report either as complete.
Git branches	main 88cb88d is the current default head; ui acd4855 contains web identity/layout adjustments; logistics 1d486e1 adds a logistics requirements document and CSS refinements.	These are shared team branches, not 12 personal branches. Review and reconcile ui/logistics into the agreed main baseline before feature integration. The inspected trees contain only web/, no mobile client or implemented server/API.

Day 1 purpose: agree scope, roles, interfaces and owners; inventory existing UI; choose website/mobile architecture; assign one reviewable output and next ticket to every contributor. Do not restart or rewrite working prototype features without evidence.

1. Day 1 sequence and team deliverables

Time-box the first day around decisions and reviewable artifacts. Adjust clock times with the team; keep the sequence.

Timebox	Team activity	Evidence
09:30-10:00	Kickoff: MVP outcome, roles, current project snapshot, decision owners.	Shared goal, attendees/owners, open questions.
10:00-11:00	Each owner inspects their committed module: implemented, simulated, missing, risky.	Short audit note linked to screens/files; no duplicate work.
11:00-12:00	Agree user journeys, permissions, status transitions, included/deferred scope.	Journey map, role/status table, decision log.
12:00-13:00	Agree web + mobile plan and API/data draft: IDs, fields, money, time, errors.	Platform decision, screen map, draft contract, unresolved fields + owners.
14:00-15:00	Create/verify individual branches; confirm integration base and review flow.	12 distinct branch names recorded; reviewer and merge process.
15:00-16:00	Write first tickets with acceptance criteria and dependencies.	Day 2 backlog; mock seams and foundation owners identified.
16:00-16:30	Readout: completed / next / blocked, with links.	Team update; every blocker has an owner and due date.

Mandatory end-of-day team packet

Artifact	Minimum contents / acceptance
Scope + decision log	MVP in/out; VIN source; payment mode; live GPS; voice/AI/quantum; service model; unknowns with owner/due date.
Journeys + permissions	Customer, seller, rider/technician, admin; status actor/transition/reason; provider contact after assignment only.
Integration draft	Bike, battery, service/slot, quote/order, appointment/job, status event fields; API owners; IDs, currency, timezone, errors, authorization.
Web + mobile delivery map	Screen ownership per client; reuse/design-system choice; supported devices; mobile framework owner and release assumptions.
Branch + backlog sheet	One personal branch per contributor; first ticket, dependency, reviewer, acceptance and evidence.

Branch audit: main, ui and logistics are the only remote heads found. Ask the maintainers to review the existing ui and logistics contributions, decide what to merge, and record main as the common integration base before all contributors create personal branches.

2. Individual Day 1 assignments - 1 of 2

Each contributor finishes with one linked artifact and a concrete Day 2 task. Pair for decisions; keep one accountable owner per deliverable.

Owner / personal branch	Day 1 work	Exact end-of-day delivery	Wait / coordinate
Harshitha - feature/harshitha-q	Frame one future optimization question (e.g. slot assignment); keep it outside production checkout/fulfillment.	1-page problem brief: objective, inputs, metric, classical baseline, data required, defer/continue recommendation.	Coordinate with Hema and Viswanath on realistic problem; research framing starts now.
Hema - feature/hema-q	Define reproducible comparison for Harshitha's candidate.	Benchmark outline: inputs/data, simple classical baseline, metric, reproducibility and stop/go criteria.	Needs candidate problem; pair today. Core MVP unaffected.
Viswanath - feature/viswanath-logistics	Draft service coverage, delivery/install, slot duration/capacity/buffer, fees, ETA, cutoff, cancellation/reschedule and out-of-area behavior.	Rules table with examples and edge cases; label assumptions; name approver.	Get quote input from Sathwik and slot/schema input from Vamsi/Pushpam. Draft begins now; approve rules by Day 2.
Amrutha - feature/amrutha-rider	Own cross-module dependency checklist. Map assigned jobs, accept/decline, job/appointment details, contact visibility and permitted status changes; inspect technician UI.	Integration matrix (owner/API/field/dependency/blocker); rider workflow/status/contact rules; mark mock-only UI actions; Day 2 integration ticket.	Start now. Real integration waits for order/appointment contract and status/permission rules; workflow/mock design proceeds.
Aakash - feature/aakash-shell	Audit current shell/routes/responsive states; define website and mobile boundaries, nav, shared UI conventions and auth handoff.	Platform decision note: web baseline, mobile target/framework recommendation, screen map, breakpoints/devices, shell tickets and risks.	Needs team framework decision and Pushpam auth/API direction; audit starts now.
Hima Bindu - feature/hima-quality	Create issue template; inspect current journeys at phone/desktop sizes; distinguish UI defects from backend/mobile gaps.	Prioritized issue board with repro, expected/actual, severity, owner, client and verification; initial blocker list.	Can audit web now; needs owners to assign fixes. Mobile verification once build exists.

Evidence beats activity: a decision note, a reproducible issue, a flow, field table, or focused branch/ticket is a valid Day 1 outcome. "Started coding" without an agreed interface is not a handoff.

3. Individual Day 1 assignments - 2 of 2

Pair customer-facing website and mobile screens by feature. Both clients must use the same approved fields, quote rules and acceptance criteria.

Owner / personal branch	Day 1 work	Exact end-of-day delivery	Wait / coordinate
Muni Sankar - feature/muni-catalog	Audit mock bike/battery/fitment data; define product fields and verified-fitment source, including compatible/incompatible/unknown.	Catalog dictionary; source/review-date workflow; cases for exact, incompatible, unknown, out-of-stock, no results; unsupported mock claims flagged.	Coordinate with Harshith and Vamsi; verified production records need a knowledgeable source/reviewer.
Sathwik - feature/sathwik-billing	Audit current total calculation; specify authoritative quote items, tax/discount/payment state.	Quote contract + worked examples; payment scope decision; mock assumptions/discrepancies; voice search remains optional.	Needs Viswanath fee rules and Pushpam API plan; formula audit starts now.
Vamsi - feature/vamsi-seller	Inventory product/seller/stock/slot/order fields; sketch persistence model and seller actions.	Draft ER diagram/schema, migrations/seed plan, seller listing/inventory/order workflow; assign shared field owners.	Needs Muni fitment model, Viswanath rules, Pushpam conventions; draft starts now.
Harshith - feature/harshith-catalog-ui	Review catalog/search/detail; define parity and empty/loading/error/unknown/out-of-stock states for both clients.	Web/mobile screen matrix; API fields needed; prototype gaps and acceptance checks.	Final integration needs Muni fields/source and Aakash conventions; mock-backed UI planning proceeds.
Pushpam - feature/pushpam-backend	Confirm no server is present in inspected tree; plan backend/auth/roles /API/environment/deploy/migration setup.	Backend decision note: stack, role authorization, API/error/version convention, config plan, first skeleton ticket and setup note.	Needs team stack and Vamsi schema direction; planning begins now.
Sathesh - feature/sathesh-checkout	Audit booking, slot UI, totals, order status and provider reveal; map same checkout on web and mobile.	Flow/screen matrix; quote/order/appointment fields; conflict/payment-failure states; note browser slot changes are simulated.	Needs Sathwik, Viswanath, Pushpam, Amrutha contracts; audit and screen map proceed.

The branch examples use one branch per person across the web/mobile work that person owns. Use the team's final naming convention consistently; do not treat existing ui as everybody's personal branch.

4. Branch rules, ownership and dependency handoffs

Everyone must have their own separate branch. One contributor, one personal feature branch, one accountable owner, reviewable changes, then PR into the agreed integration base. Feature work does not go directly onto main.

Rule	Team convention
Base branch	Restore/sync source first. Confirm whether main is protected integration base and whether ui is active. Pull latest agreed base before branching. Do not assume ui is a personal branch or default base.
Branch name	feature/<first-name>-<module>, e.g. feature/amrutha-integration-rider. Record 12 unique names. One branch per person even if their work spans web and mobile.
Commits + PR	Keep commits focused and prototype running. Name a reviewer from an adjacent module. Put contract changes, screenshots/demo and acceptance evidence in PR.
Shared contracts	Named owners maintain the authoritative API/schema document. Other changes are proposed in reviewed, versioned updates.
Integration	Merge small increments after interfaces are agreed. Clearly label mock adapters; feature owners own their acceptance checks.

Who owns the handoff

Artifact	Accountable owner(s)	Consumers
Catalog / fitment	Muni Sankar	Harshith, Vamsi, Pushpam, Amrutha.
Service area / fee / slots / ETA	Viswanath	Sathwik, Vamsi, Sathesh, Amrutha, Pushpam.
Quote rules / order API / persistence	Sathwik (money), Pushpam (API), Vamsi (data)	Sathesh, Amrutha, seller/technician owners.
Job / appointment / status / contact	Amrutha (workflow), Viswanath (service rules), Pushpam (authorization/API)	Sathesh, Vamsi, Hima.
Web/mobile shell + conventions	Aakash	Harshith, Sathesh and all client contributors.
Defect and release evidence	Hima Bindu	Every feature owner verifies own fix.

What can start now: audits, scope/flow documents, branch setup, UI states, mock-backed design and contract drafts. **What must wait to integrate:** production catalog waits for verified fitment fields/source; final quote waits for agreed fee/tax rules; confirmed booking waits for quote and slot rules; rider assignment waits for persisted order/appointment records plus authorized status transitions. Escalate a blocker after one work session with the exact needed decision/input.

5. Build website and mobile in one repository

Yes - one repository is recommended for this team. Use a monorepo with separate clients and one shared API contract/backend. The existing Vite web app stays in web/; build the actual Android/iOS app in mobile/; add backend/ and contracts/ as those components are created. A single repository makes coordinated changes and reviews easier while preserving independent client code.

The current phone-sized preview is responsive web, not an installable mobile app. Share API schemas, sample fixtures, product terminology, design tokens and acceptance criteria. Keep web and mobile presentation code separate unless the team deliberately selects a single cross-platform UI framework.

Target architecture to agree on Day 1

Layer	Website	Mobile app	Shared rule
Client foundation	Continue Vite/vanilla JS as starting web baseline; routes, role navigation, responsive layout.	Create actual Android/iOS project on the framework chosen Day 1; Aakash owns app shell/navigation.	Align role model, design tokens, screen names, errors and accessibility; UI code can differ.
Customer journey	Bike profile, catalog/search/detail, quote/cart, address/slot, confirmation, status/history.	Same essential flow, touch-first. Add camera/push only after requirements/privacy decisions.	Same API fields, fitment outcome, amount/status semantics; feature owner maintains parity checklist.
Operations	Vendor and technician/rider role surfaces responsive on web initially.	Customer app first; decide later if technician needs dedicated field app.	Backend enforces roles/ownership; contact only after assignment.
Backend	No backend found in inspected commit; call authenticated API; isolate mock adapter for dev.	Call same API; never recalculate authoritative price or booking rules locally.	Pushpam API/auth/deploy; Vamsi migrations; versioned schema/OpenAPI and mock examples.
Trust/safety	Verified/unknown fitment; server quote; clear conflicts/failures.	Same rules, network/offline handling and protected session.	UTC storage/local display, authorization, idempotency, valid transition checks.

Build sequence

Stage	Website	Mobile	Exit evidence
Day 1-2	Audit, screen/routes ownership, responsive rules.	Choose platform; target OS/device; map journeys.	Decision note, screen parity map, data/API draft.
Weeks 1-2: foundation	Clean checkout runs; shell, route guards, API client boundary, responsive baseline.	Installable shell, nav, environment and API client.	Both clients launch; mock adapter isolated; sample roles.
Weeks 2-4: catalog	Bike, fitment-aware search/detail and all loading/error/unknown states.	Equivalent bike/catalog/detail, touch-friendly.	Same seeded records return the same fitment result in both.
Weeks 4-6: book	Quote, slot, order, confirmation and tracking.	Same journey and server totals.	Order with exact server total; conflict cannot double-book.
Weeks 5-8: fulfill/release	Seller response, technician panel, customer status; responsive hardening.	Status updates/notifications as approved; device/network review.	Authorized transitions; acceptance passes on web + supported phones.

Framework note: keep the existing Vite website. Choose one mobile framework after checking team skills, target devices and build access. If Flutter is chosen, it is new client work; "Flutter-ready" in README does not mean a Flutter implementation exists. Avoid teams choosing incompatible mobile frameworks independently.

6. Dependencies and decisions to settle

Do not idle on a dependency. Continue the independent artifact or mock path, and label exactly which integration step is blocked.

Work	Can begin now?	Needed for production integration	Owner(s)
Scope / project audit / issue list	Yes	Name product decision owner; close disagreements.	All; Hima tracks.
Shell / route map	Yes	Agree web/mobile framework and role/screen boundaries before implementation.	Aakash.
Fitment UI/data	Audit + mock states yes	Fields, source, review date; exact/unknown/incompatible rules.	Muni + Harshith.
Quote/booking UI	Audit + mock breakdown yes	Fee/tax/payment/order contract; slot capacity and hold/conflict rules.	Sathwik + Sathesh + Viswanath.
Backend/schema	Design now	Field ownership, IDs, money/time/error conventions; then auth and migrations.	Pushpam + Vamsi.
Rider workflow	Screens/mock jobs yes	Order + appointment/job records, provider identity, transition permissions/API.	Amrutha + Pushpam + Viswanath.
GPS / voice / quantum	Decision/research only	Consent/provider reliability/search stability; otherwise status updates/basic search.	Amrutha/Pushpam; Sathwik; Harshitha/Hema.

Unresolved decisions must have an owner and due date

Decision	Owner to name	Target / safe interim assumption
Mobile framework + OS/devices	Aakash + team lead	Day 1; one cross-platform codebase if skills/build access allow.
MVP scope authority	Team lead named at kickoff	Day 1; changes recorded with reason and displaced work.
VIN lookup source	Muni + product reviewer	Day 1; manual bike selection until lookup is verified.
Payment provider or sandbox/mock	Sathwik + Pushpam	Day 1; no live payments before provider/security requirements are agreed.
Coverage, fees, slot capacity	Viswanath	Draft Day 1; team approval by Day 2.
Contract owner/location/version	Pushpam + Vamsi + Amrutha	Owner and draft Day 1; usable shared contract by Day 3.
Admin scope and release clients	Product decision owner + Aakash	Day 1; role scope explicit even if admin UI follows customer slice.

7. Day 1 closeout acceptance checklist

Day 1 is complete when these items are visible and reviewable. An open decision can stay open if its owner and deadline are recorded.

Check	Pass condition
Project snapshot	Prototype is distinguished from mock behavior; backend/mobile marked absent in inspected tree; all three remote branches and their current heads have an integration owner.
12 owners	Each person has accountable output plus reviewer/partner for cross-module decisions.
Separate branches	12 unique personal branches on agreed base (or a named restoration blocker and owner); no direct feature work on main.
Web + mobile	Website, app, shared backend and screen scope explicit; framework/platform owner and decision recorded.
Scope + journey	Included/deferred items, user roles, unknown-fitment handling, provider visibility, status authority recorded.
Contract draft	Core entities/API boundaries have owners; money, timezone, IDs, auth, errors and transitions noted.
Next tickets	Each contributor has one concrete ticket with acceptance, dependency and evidence.
Actionable blockers	Input needed, provider, affected work and required date recorded.
Status update	Each posts done / next / blocked and links artifact, PR, issue or demo.

End-of-day update template

Owner: name / module / branch

Completed: link to audit, decision, flow, schema, issue, PR or demo

Next: one concrete Day 2 outcome

Blocked: exact input needed, from whom, by when (or "none")

Clients: website / mobile / backend / shared

Day 2 recommended focus

Approve user journeys, status/permission table, service/fee/slot rules, mobile platform choice and API/data contract owner. Begin foundation implementation once the first contract draft is visible. Let web and app screens proceed in parallel with labeled mock fixtures, then replace mock adapters with the same backend API.

Audit note

This revision uses the supplied eight-page guide and inspected remote trees at main 88cb88d, ui acd4855 and logistics 1d486e1 on 09 Oct 2026. The actual implementation is a responsive Vite/vanilla JavaScript web prototype backed by browser-side mock data; the mobile app and backend/API remain planned work. The README and logistics document were treated as specifications, not proof of implemented behavior. Re-audit the branches before implementation because remote heads can change.